

the same machine and running on port 8080, access the admin interface by using `localhost:8080/sitewhere/admin` and the REST interface using `localhost:8080/sitewhere`. The default credential for both the interfaces is `admin:password`.

## Device events

SiteWhere supports the following device events through the MQTT protocol using JSON or Google's Protobuf format. In this article we'll use the JSON format and publish the payloads to the MQTT topic `SiteWhere/input/json` as per the tenant configuration.

**Registration:** Any device should be registered initially by publishing the following JSON payload:

```
{
  "hardwareId": "123-MyRpiSense-4567890",
  "type": "RegisterDevice",
  "request": {
    "hardwareId": "123-RpiSense-4567890",
    "specificationToken": "xxxx-xx-xx-xx-xxxxxx",
    "siteToken": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx"
  }
};
```

Here `hardwareId` can be any unique value; `siteToken` identifies the preferred site, say construction site, and `specificationToken` identifies Raspberry Pi from the device specifications through the *Admin* interface.

You can identify the new devices under the *Admin* UI in the *Devices* section and *Site* view section. You can also register a device manually here.

**Location update:** This helps in geo mapping your device initially or whenever the device is moved for better location awareness in an associated site. To do this, publish the following payload with the suitable parameters:

```
{
  "hardwareId": "123-MyRpiSense-4567890",
```

```
  "type": "DeviceLocation",
  "request": {
    "latitude": "18.5204",
    "longitude": "73.8567",
    "elevation": "560",
    "updateState": false,
    "eventDate": "2017-06-01T12:00:00.390Z"
  }
}
```

**Sending measurements:** To send measured values from a device like Raspberry Pi, let's look at publishing system metrics like the load average of the CPU and memory usage, periodically. In Node.js and NodeRED environments, these can be calculated with the following code, considering `avg1` and `avg5` as the load average for the last 1 minute and 5 minutes, respectively, and `pmem` as memory utilisation. You may also add the average of the last 15 minutes as one more property.

```
const os=require('os');
var avg1 = os.loadavg()[0];
var avg5 = os.loadavg()[1];
var pmem=(100 - (os.freemem()/os.totalmem())*100).toFixed(2);
```

Now form the payload in the following format, and publish periodically or on an event basis.

```
{
  "hardwareId": "123-MyRpiSense-4567890",
  "type": "DeviceMeasurements",
  "request": {
    "measurements": {
      "rpi.loadavg1": avg1,
      "rpi.loadavg5": avg5,
      "rpi.usagemem": pmem,
    },
    "updateState": true,
    "eventDate": now
  }
};
```

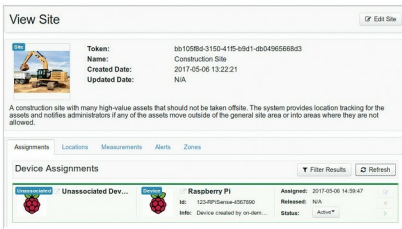


Figure 2: Admin interface – site information and registered devices

If you have a suitable target board and sensors, you can send better payloads, say, weather metrics like temperature, humidity, pressure, etc, or motion sensing metrics like roll, pitch and yaw from an accelerometer's or gyroscope's inputs. The SenseHAT add-on board for Raspberry Pi is a good choice for these sensors.

**Sending alerts:** A device can send alerts when anomalies are detected in measured values by publishing the following payload: